

Skill session-4

Task 1 : To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

```
vennela_07@VennelaPC:~$ mkdir token_parser
vennela_07@VennelaPC:~$ cd token_parser
vennela_07@VennelaPC:~/token_parser$ nano token_parser.c
vennela_07@VennelaPC:~/token_parser$ gcc token_parser.c -o token_parser
vennela_07@VennelaPC:~/token_parser$ ./token_parser
Enter input: Hello, world;

--- Debug: Original Input ---
Hello, world;

--- Tokens ---
Token 1: Hello
Token 2: world

--- Token Stream Validation ---
Valid: 2 tokens identified.

--- Debug Parsing Output ---
tokens[0].value = "Hello"
tokens[1].value = "world"
vennela_07@VennelaPC:~/token_parser$ ./token_parser
Enter input: apple,banana;orange mango

--- Debug: Original Input ---
apple,banana;orange mango

--- Tokens ---
Token 1: apple
Token 2: banana
Token 3: orange
Token 4: mango

--- Token Stream Validation ---
Valid: 4 tokens identified.

--- Debug Parsing Output ---
tokens[0].value = "apple"
tokens[1].value = "banana"
tokens[2].value = "orange"
tokens[3].value = "mango"
```

```

        token = strtok(NULL, delimiters);
    }

    printf("\n--- Token Stream Validation ---\n");

    if (count == 0)
    {
        printf("Invalid: No tokens found.\n");
    }
    else
    {
        printf("Valid: %d tokens identified.\n", count);
    }

    printf("\n--- Debug Parsing Output ---\n");

    for (i = 0; i < count; i++)
    {
        printf("tokens[%d].value = \"%s\"\n",
              i, tokens[i].value);
    }

    return 0;
}

```

Task 2 : To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.

```

vennela_07@VennelaPC:~/token_parser$ mkdir parser_logic
cd parser_logic
vennela_07@VennelaPC:~/token_parser/parser_logic$ pwd
/home/vennela_07/token_parser/parser_logic
vennela_07@VennelaPC:~/token_parser/parser_logic$ nano parser.c
vennela_07@VennelaPC:~/token_parser/parser_logic$ gcc parser.c -o parser
vennela_07@VennelaPC:~/token_parser/parser_logic$ gcc parser.c -o parser
vennela_07@VennelaPC:~/token_parser/parser_logic$ ./parser
Enter command: COPY file1.txt file2.txt

--- Parser Output ---
Valid COPY command.

--- Parse Tree ---
|-- COPY
|  |-- file1.txt
|  |-- file2.txt

--- Execution Structure ---
Operation : COPY
Source    : file1.txt
Destination: file2.txt

Parsing completed successfully.

```

```

Parsing completed successfully.
vennela_07@VennelaPC:~/token_parser/parser_logic$ ./parser
Enter command: DELETE file1.txt

--- Parser Output ---
Valid DELETE command.

--- Parse Tree ---
|-- DELETE
    |-- file1.txt

--- Execution Structure ---
Operation: DELETE
File      : file1.txt

Parsing completed successfully.

```

```

GNU nano 7.2                                parser.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_INPUT 200
#define MAX_TOKENS 10

typedef struct Node
{
    char value[50];
    struct Node *left;
    struct Node *right;
} Node;

/* Create a new parse tree node */
Node *createNode(char *value)
{
    Node *newNode = (Node *)malloc(sizeof(Node));

    strcpy(newNode->value, value);
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}

/* Display parse tree */
void printTree(Node *root, int level)
{
    int i;

    if (root == NULL)
        return;

    for (i = 0; i < level; i++)
        printf(" ");

    printf("|-- %s\n", root->value);

    printTree(root->left, level + 1);
    printTree(root->right, level + 1);
}

```

```
GNU nano 7.2                                parser.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_INPUT 200
#define MAX_TOKENS 10

typedef struct Node
{
    char value[50];
    struct Node *left;
    struct Node *right;
} Node;

/* Create a new parse tree node */
Node *createNode(char *value)
{
    Node *newNode = (Node *)malloc(sizeof(Node));

    strcpy(newNode->value, value);
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}

/* Display parse tree */
void printTree(Node *root, int level)
{
    int i;

    if (root == NULL)
        return;

    for (i = 0; i < level; i++)
        printf(" ");

    printf("|-- %s\n", root->value);

    printTree(root->left, level + 1);
    printTree(root->right, level + 1);
}
```

```
}

/* Free memory */
void freeTree(Node *root)
{
    if (root == NULL)
        return;

    freeTree(root->left);
    freeTree(root->right);

    free(root);
}

int main()
{
    char input[MAX_INPUT];
    char *tokens[MAX_TOKENS];
    int count = 0;
    int i;

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);

    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';

    /* Handle empty command */
    if (strlen(input) == 0)
    {
        printf("\nEmpty command detected.\n");
        printf("Nothing to execute.\n");
        return 0;
    }

    /* Tokenize input */
    char *token = strtok(input, " \t");

    while (token != NULL && count < MAX_TOKENS)
    {
        tokens[count] = token;
        count++;

        token = strtok(NULL, " \t");
    }
}
```

```
}

printf("\n--- Parser Output ---\n");

/* Validate command */
if (strcmp(tokens[0], "COPY") == 0)
{
    if (count != 3)
    {
        printf("Syntax Error: COPY requires 2 arguments.\n");
        printf("Correct format: COPY source destination\n");
        return 1;
    }

    printf("Valid COPY command.\n");

    Node *root = createNode("COPY");
    root->left = createNode(tokens[1]);
    root->right = createNode(tokens[2]);

    printf("\n--- Parse Tree ---\n");
    printTree(root, 0);

    printf("\n--- Execution Structure ---\n");
    printf("Operation : COPY\n");
    printf("Source      : %s\n", tokens[1]);
    printf("Destination: %s\n", tokens[2]);

    freeTree(root);
}
else if (strcmp(tokens[0], "DELETE") == 0)
{
    if (count != 2)
    {
        printf("Syntax Error: DELETE requires 1 argument.\n");
        printf("Correct format: DELETE filename\n");
        return 1;
    }

    printf("Valid DELETE command.\n");

    Node *root = createNode("DELETE");
    root->left = createNode(tokens[1]);

    printf("\n--- Parse Tree ---\n");
    printTree(root, 0);

    printf("\n--- Execution Structure ---\n");
    printf("Operation: DELETE\n");
    printf("File       : %s\n", tokens[1]);

    freeTree(root);
}
else
{
    printf("Syntax Error: Unknown command '%s'.\n", tokens[0]);
    printf("Supported commands: COPY, DELETE\n");
    return 1;
}

printf("\nParsing completed successfully.\n");

return 0;
}
```